```
x = 5

do (x) ->
  x = 3
  console.log x

console.log x

# 3
# 5
```

Using a do expression, you can have an explicit, local variable that can't clobber any outer variable when assigned.

This approach gives you a new way of writing a function that has variable names that shadow outer scopes:

```
shadowing = do (x) -> (y) ->
  x = 3
  x + y

shadowing 5
# 8
```

If you use a `do` expression in this way, then either there must be an outer variable that you want to shadow by assignment inside the `do` expression or you must assign a value in the parameters for the `do` expression:

```
do (notPreviouslyDefined) -> notPreviouslyDefined = 9
# ReferenceError: notPreviouslyDefined is not defined

do (notPreviouslyDefined='') -> notPreviouslyDefined = 'It is now'
# 'It is now'
```

Outside the `do` expression, the variable is still not defined:

```
notPreviouslyDefined
# ReferenceError: notPreviouslyDefined is not defined
```

Unfortunately, the syntax to achieve variables that shadow is a little clunky by CoffeeScript standards. Still, given that function parameters shadow the `do` expression, you can have local variables and can even approximate a future feature of JavaScript called `let` that you'll learn about in chapter 13. Until then, that's all there is on syntax.